Python – Collections, functions and Modules

ACCESSING LIST:

Understanding how to create and access elements in a list.

-> In Python, a list is created by placing elements inside square brackets [], separated by commas. You access an element by its position (index) using square brackets.

- **Creation:** my_list = ["apple", "banana", "cherry"]
- **Accessing:** print (my_list[0]) would output "apple".

Indexing in lists (positive and negative indexing).

-> Python uses **Zero-based indexing**, meaning the first item is at index 0.

- **Positive Indexing:** Starts from the beginning.
    - [0] is the 1st element.
    - [1] is the 2nd element.
- **Negative Indexing:** Starts from the end, which is very useful when you don't know the length of the list.
    - [-1] is the last element.
    - [-2] is the second-to-last element.

Slicing a list: accessing a range of elements.

-> Slicing allows you to get a sub-section of a list. The syntax is list[start:stop:step].

- **Start:** The index where the slice begins (inclusive).
- **Stop:** The index where the slice ends (exclusive—it does **not** include the element at this index).
- **Step:** (Optional) The increment between elements (e.g., a step of 2 takes every second item).

Examples:

- numbers = [0, 10, 20, 30, 40, 50]

- numbers[1:4] returns [10, 20, 30] (indices 1, 2, and 3).
- numbers[:3] returns [0, 10, 20] (starts from the beginning).
- numbers[3:] returns [30, 40, 50] (goes to the very end).
- numbers[::2] returns [0, 20, 40] (takes every second element).

LIST OPERATIONS:

Common list operations: concatenation, repetition, membership.

-> Concatenation

The process of joining two or more sequences end-to-end to create a single, new sequence. In Python, this is performed using the addition operator (+). It does not modify the original lists but produces a third list containing all elements from the operands.

Repetition

The process of creating a new list by duplicating the elements of a source list a specified number of times. This is performed using the multiplication operator (*).

Membership

A Boolean operation used to determine whether a specific element exists within a collection. Using the in or not in keywords, Python performs a linear search through the list, returning true if a match is found.

Understanding list methods like append (), insert (), remove (), pop ().

-> Methods are functions tied to the list class that primarily modify the list **in-place** (meaning they change the original list rather than creating a new one).

append (object) - A method that adds a single item to the end of the list.

Example:

```
numbers = [1, 2, 3]

numbers.append(4)

print(numbers)                    Output: [1, 2, 3, 4]
```

insert (index, object) - A method that places an element at a specific position defined by an index.

Example:

```
numbers = [1, 2, 3]

numbers.insert(1, 10)   # insert 10 at index 1

print(numbers)                    Output: [1, 10, 2, 3]
```

remove (value) - A method that searches for the first occurrence of a specific value and deletes it from the list.

Example:

```
numbers = [1, 2, 3, 2]

numbers.remove(2)

print(numbers)                    Output: [1, 3, 2]
```

pop([index]) - A method that removes the element at a given position and returns that element as a value.

Example:

```
numbers = [1, 2, 3, 4]

x = numbers.pop()     # removes last element

print(x)

print(numbers)                    Output: [1, 2, 3]
```

WORKING WITH LISTS:

Iterating over a list using loops.

Iteration means going through each element of a list one by one using loops like for or while.

Sorting and reversing a list using sort(), sorted(), and reverse().

sort() → sorts the original list

sorted() → returns a new sorted list

reverse() → reverses the list

Basic list manipulations: addition, deletion, updating, and slicing.

Addition → using append() or insert()

Example:

```
fruits = ["apple", "banana"]
# add at end
fruits.append("mango")
# add at specific position
fruits.insert(1, "orange")
print(fruits)                    Output: ['apple', 'orange', 'banana', 'mango']
```

Deletion → using remove() or pop()

Example:

```
fruits = ["apple", "banana", "mango", "banana"]
# remove by value
fruits.remove("banana")
# remove by index
fruits.pop(1)
print(fruits)                    Output: ['apple', 'banana']
```

Updating → changing value using index

Example:

```
fruits = ["apple", "banana", "mango"]
# update element
fruits[1] = "orange"
print(fruits)                    Output: ['apple', 'orange', 'mango']
```

Slicing → accessing part of list

Example:

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])   # index 1 to 3
```

```
print(numbers[:3])    # from start to index 2
print(numbers[2:])    # from index 2 to end
print(numbers[::-1])  # reverse list
```

Output:

```
[20, 30, 40]
[10, 20, 30]
[30, 40, 50]
[50, 40, 30, 20, 10]
```

TUPLE:

Introduction to tuples, immutability.

A tuple is a collection of elements similar to a list, but it is immutable, which means its elements cannot be changed after creation. Defined using parentheses (), Ordered and allows duplicate values, Cannot be modified (no add/remove).

Creating and accessing elements in a tuple.

Tuples are created using parentheses and elements are accessed using index values.

Example:

```
# creating a tuple
numbers = (10, 20, 30, 40)

# tuple with different data types
data = ("Priyanshu", 21, 5.9)
print(numbers[2])
print(numbers[-2])

print(data)
```

Output:

30

30

('Priyanshu', 21, 5.9)

Basic operations with tuples: concatenation, repetition, membership.

- Concatenation (+) → combines tuples
  Example:
  t1 = (1, 2, 3)
  t2 = (4, 5, 6)
  t3 = t1 + t2
  print(t3) Output: (1, 2, 3, 4, 5, 6)

- Repetition (*) → repeats tuple
  Example:
  t1 = (1, 2)
  t2 = t1 * 3
  print(t2) Output: (1, 2, 1, 2, 1, 2)

- Membership (in / not in) → checks element
  Example:
  t1 = (10, 20, 30, 40)
  print(20 in t1) # check if present
  print(50 in t1)

  print(30 not in t1) # check if not present

  Output:
  True
  False
  False

ACCESSING TUPLES:

Accessing tuple elements using positive and negative indexing.

- Tuple elements are accessed using index values.
- Positive indexing starts from 0 (left to right)

- Negative indexing starts from -1 (right to left)

Slicing a tuple to access ranges of elements.

Slicing is used to access a range of elements from a tuple.

Example:

```
numbers = (10, 20, 30, 40, 50, 60)

# slicing from index 1 to 4 (excluding 4)
print(numbers[1:4])

# from start to index 3
print(numbers[:4])

# from index 2 to end
print(numbers[2:])

# using step
print(numbers[0:6:2])

# reverse tuple
print(numbers[::-1])
```

Output:

```
(20, 30, 40)
(10, 20, 30, 40)
(30, 40, 50, 60)
(10, 30, 50)
(60, 50, 40, 30, 20, 10)
```

DICTIONARIES:

Introduction to dictionaries: key-value pairs.

A dictionary is a collection of data stored in key-value pairs.

- Defined using curly braces {}.
- Each key is unique
- Values can be of any data type

Example:

```
student = {
   "name": "Priyanshu",
   "age": 21,
   "course": "B,E"
}


print(student)
```

Output:

{'name': 'Priyanshu', 'age': 21, 'course': B.E}

Accessing, adding, updating, and deleting dictionary elements.

- Accessing → using key
  Example:
  ```
  student = {"name": "Priyanshu", "age": 21}
  print(student["name"])   # using key
  ```

  Output:
  Priyanshu

- Adding → assign new key-value
  Example:
  ```
  student = {"name": "Priyanshu", "age": 21}
  ```

```
# add new key-value
student["city"] = "Surat"
print(student)
```

Output:
{'name': 'Priyanshu', 'age': 21, 'city': 'Surat'}

- Updating → change existing value
  Example:
  ```
  student = {"name": "Priyanshu", "age": 21}

  # update value
  student["age"] = 22
  print(student)
  ```

  Output:
  {'name': 'Priyanshu', 'age': 22}

- Deleting → using del or pop()
  Example:
  ```
  student = {"name": "Priyanshu", "age": 21, "course": "BCA"}

  student.pop("age")      # remove by key
  print(student)
  ```

  Output:
  {'name': 'Priyanshu'}

Dictionary methods like keys (), values (), and items ().

- keys() → returns all keys
- values() → returns all values
- items() → returns key-value pairs
  Example:
  ```
  student = { "name": "Priyanshu", "age": 21, "course": "B.E"}
  print(student.keys())
  print(student.keys())
  print(student.items())
  ```

  Output:

dict_keys(['name', 'age', 'course'])

dict_values(['Priyanshu', 21, B.E])

dict_items([('name', 'Priyanshu'), ('age', 21), ('course', B.E)])

WORKING WITH DICTIONARIES:

Iterating over a dictionary using loops.

We can iterate through a dictionary using a for loop to access keys, values, or both.

Example:

```
student = {
   "name": "Priyanshu", "age": 21,  "course": "B.E"
}
for key in student:
   print(key)
```

Output:

name

age

course

Merging two lists into a dictionary using loops or zip ().

Two lists can be combined into a dictionary where one list acts as keys and the other as values.

Example:

```
keys = ["name", "age", "course"]
values = ["Priyanshu", 21, "B.E"]
result = dict(zip(keys, values))
print(result)
```

Output:

{'name': 'Priyanshu', 'age': 21, 'course': 'B.E'}

Counting occurrences of characters in a string using dictionaries.

We can use a dictionary to count how many times each character appears in a string.

Example:

```
text = "programming"
count = {}

for ch in text:
    count[ch] = count.get(ch, 0) + 1
print(count)
```

Output:

{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}

FUNCTIONS:

Defining Functions in Python

A function is a reusable block of code used to perform a specific task.

Syntax:

```
def function_name(parameters):
    # body of function
    return value
```

Example:

```
def greet():
    print("Hello")
```

Types of Functions.

(i) Function without parameters and without return value

```
def greet():
```

```
    print("Hello, welcome!")
greet()
```

Output:

Hello, welcome!

(ii) Function with parameters and without return value

```
def add(a, b):
    print("Sum is:", a + b)
add(5, 3)
```

Output:

Sum is: 8

(iii) Function without parameters and with return value

```
def get_number():
    return 10
x = get_number()
print(x)
```

Output:

10

(iv) Function with parameters and with return value

```
def multiply(a, b):
    return a * b

result = multiply(4, 5)
print(result)
```

Output:

20

Anonymous Functions (Lambda Functions).

A lambda function is a small anonymous function with one expression.

Syntax:

lambda arguments : expression

Example:

```
square = lambda x: x * x
print(square(6))
```

Output: 36

MODULES:

Introduction to Python Modules and Importing Modules

A module is a file containing Python code (functions, variables, etc.) that can be reused in another program.

Modules help in:

- Code reusability
- Better organization

Importing a module:

Example:

```
import math
print(math.sqrt(25))
```

Output: 5

Standard Library Modules: math, random

(i) math module

Used for mathematical operations.

```
import math
print(math.sqrt(25))     # square root
print(math.pow(2, 3))    # power
```

```
print(math.factorial(5)) # factorial
print(math.pi)          # value of π
```

Output:

5.0

8.0

120

3.141592653589793

(ii) random module

Used to generate random values.

```
import random

print(random.randint(1, 10))   # random integer
print(random.random())        # random float (0 to 1)
print(random.choice([10, 20, 30]))  # random element
```

Output:

7

0.4567

20

Creating Custom Modules

You can create your own module by saving functions in a .py file.